Parcial 2

Santiago José Rodríguez Gutiérrez
Nicolas Jacobo Mondragón Rey
Nicolas Emilio Morantes Puentes

Universidad Sergio arboleda

Inteligencia artificial

Sanchez Cifuentes Joaquin Fernando

15 de octubre de 2026

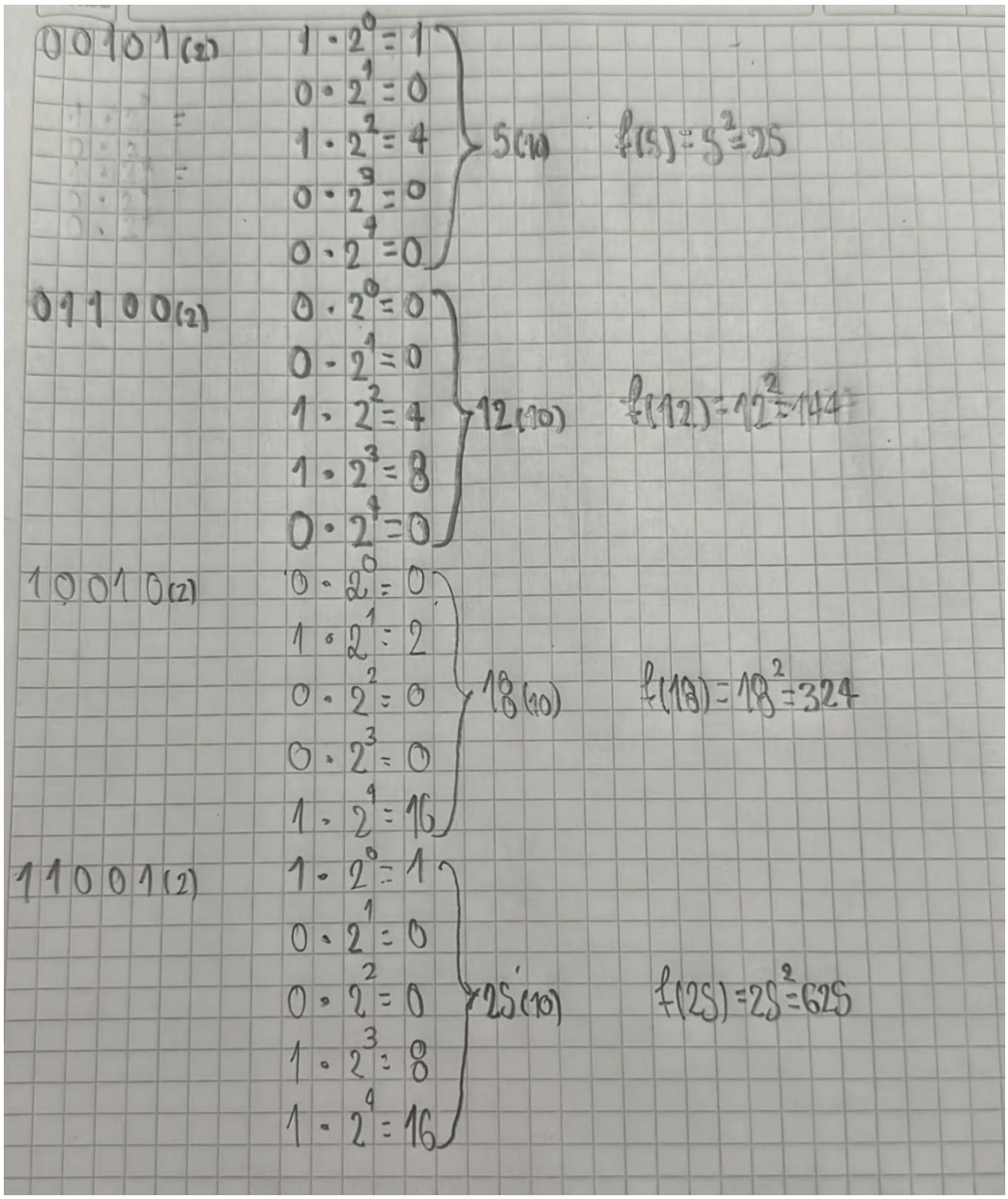
Contenido

Punto 1.....3
Punto 2.....4
Punto 3.....6
Punto 4.....9
Punto 5.....13

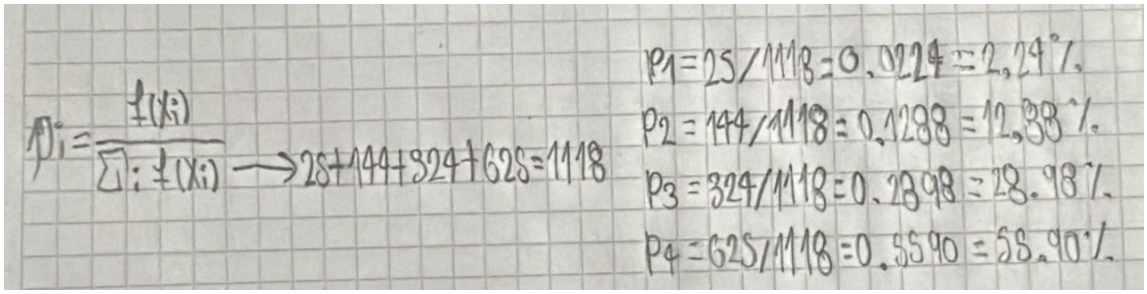
3

Punto 1

1.

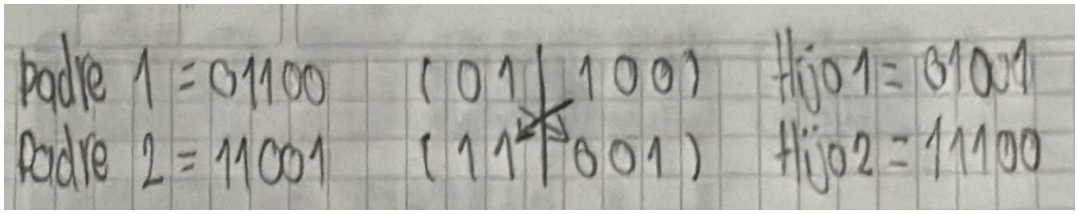

00101(2)
1·2^0=1
0·2^1=0
1·2^2=4
0·2^3=0
0·2^4=0
5(10)
f(5)=5^2=25
01100(2)
0·2^0=0
0·2^1=0
1·2^2=4
1·2^3=8
0·2^4=0
12(10)
f(12)=12^2=144
10010(2)
0·2^0=0
1·2^1=2
0·2^2=0
0·2^3=0
1·2^4=16
18(10)
f(18)=18^2=324
11001(2)
1·2^0=1
0·2^1=0
0·2^2=0
1·2^3=8
1·2^4=16
25(10)
f(25)=25^2=625

2.

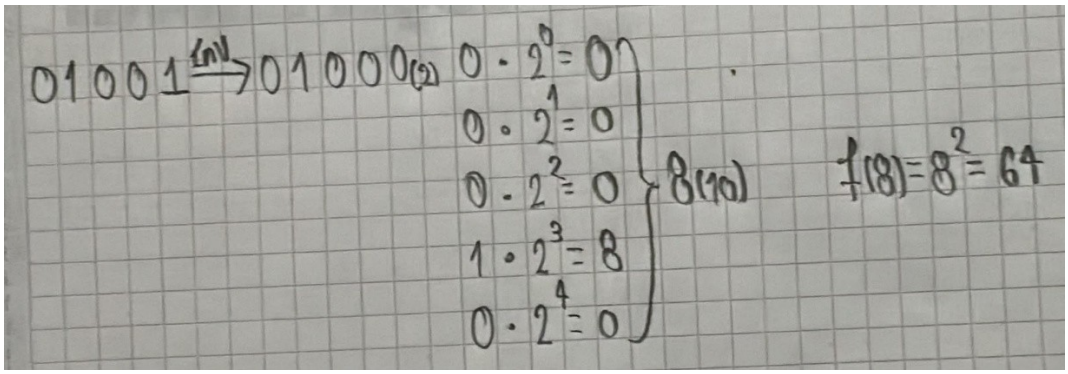

pi=f(xi)/Σ f(xi) → 25+144+324+625=1118
p1=25/1118=0.0224=2,24%
p2=144/1118=0.1288=12,88%
p3=324/1118=0.2898=28.98%
p4=625/1118=0.5590=55.90%

3.

4



4.



5. **¿Qué relación existe entre un algoritmo evolutivo y un algoritmo genético?**

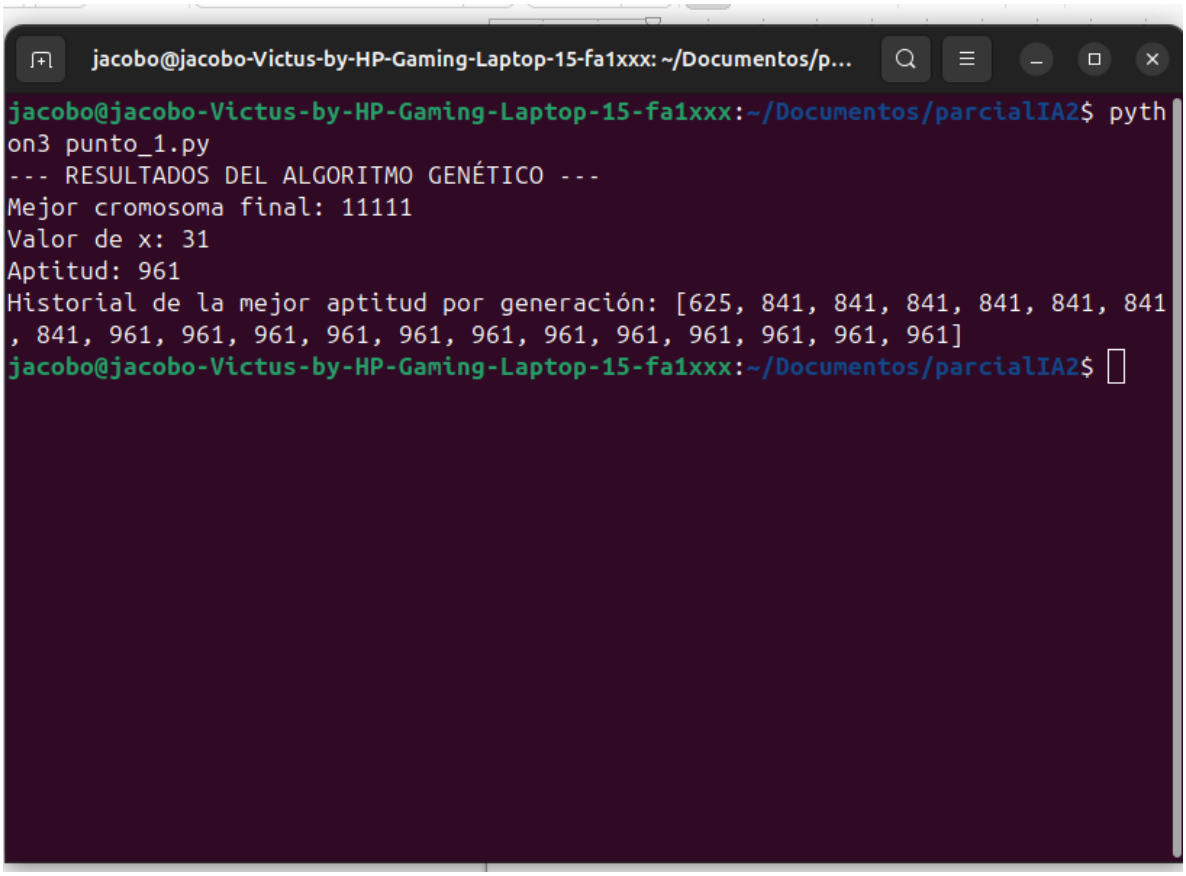
Un algoritmo evolutivo es el término general para cualquier sistema de optimización inspirado en la evolución natural (como la selección natural y la genética), en cambio un algoritmo genético es un tipo específico y muy popular de algoritmo evolutivo que utiliza codificación basada en cromosomas (como cadenas binarias) y operadores genéticos concretos (cruce, mutación y selección)

¿Qué función cumplen selección, cruce y mutación?

Selección: Elige a los individuos más aptos para reproducirse así se fomenta la explotación de buenas soluciones
Cruce: Combina bloques genéticos de dos padres para generar descendientes que potencien las mejores características de ambos
Mutación: Introduce cambios aleatorios para inyectar diversidad en la población y evitar estancamientos

¿Por qué una mutación puede ser útil aunque empeore la aptitud inmediata?
Porque evita que el algoritmo quede atrapado en mínimos o máximos locales, ya que empeorar temporalmente la aptitud, permite explorar nuevas regiones del espacio de búsqueda que de otro modo serían inaccesibles, pudiendo conducir a mejores soluciones a largo plazo

Ejecución



Punto 2

A. Componente matemático y conceptual

Regla: moverse al vecino de mayor f(x) solo si f(vecino) > f(actual).

1. Desde $x_0 = 0$

x=0 (f=1): vecino f(1)=4 → 4 > 1, se mueve a 1.
x=1 (f=4): vecinos f(0)=1 y f(2)=7 → 7 > 4, se mueve a 2.
x=2 (f=7): vecinos f(1)=4 y f(3)=5 → ninguno > 7, se detiene.
Respuesta: 0 → 1 → 2, final x = 2, f = 7.

2. Desde $x_0 = 4$

x=4 (f=3): vecinos f(3)=5 y f(5)=6 → el mayor es 6, se mueve a 5.
x=5 (f=6): vecinos f(4)=3 y f(6)=9 → 9 > 6, se mueve a 6.
x=6 (f=9): vecinos f(5)=6 y f(7)=8 → ninguno > 9, se detiene.
Respuesta: 4 → 5 → 6, final x = 6, f = 9.

3. Máximos

Locales: x=2 (f=7) y x=6 (f=9).

Global: x=6 (f=9).

4. Efecto del estado inicial

Cada x_0 sube al máximo más cercano: {0,1,2,3} → 2 y {4,…,8} → 6.

5. Estrategia

Reinicios aleatorios. Costo: k veces más evaluaciones de f.

B. Implementación en Python

```
VALORES = [1, 4, 7, 5, 3, 6, 9, 8, 5]

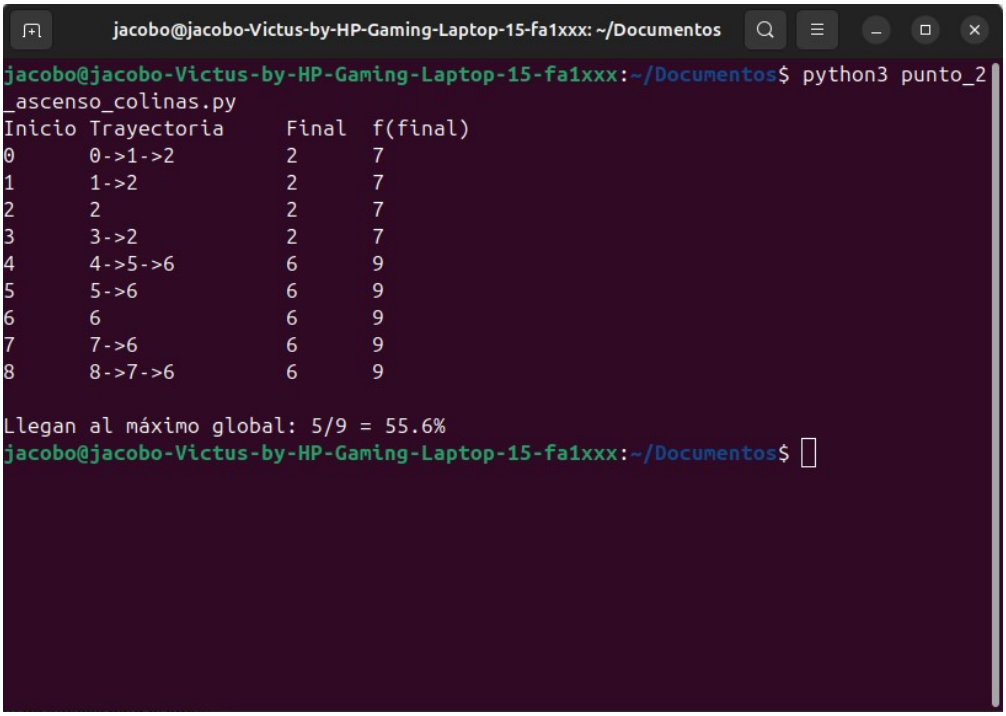

def ascenso_colinas(valores, inicio):
    """Devuelve el estado final y la trayectoria."""
    x = inicio
    trayectoria = [x]
    while True:
        vecinos = [v for v in (x - 1, x + 1) if 0 <= v < len(valores)]
        mejor = max(vecinos, key=lambda v: valores[v])
        if valores[mejor] > valores[x]:  # solo mejora estricta
            x = mejor
            trayectoria.append(x)
        else:
            return x, trayectoria


if __name__ == "__main__":
    print(f"{'Inicio':<7}{'Trayectoria':<16}{'Final':<7}{'f(final)':<8}")
    global_max = max(VALORES)
    aciertos = 0
    for inicio in range(len(VALORES)):
        final, tray = ascenso_colinas(VALORES, inicio)
        print(f"{inicio:<7}{'->'.join(map(str, tray)):<16}{final:<7}{VALORES[final]:<8}")
        aciertos += VALORES[final] == global_max
    n = len(VALORES)
    print(f"\nLlegan al máximo global: {aciertos}/{n} = {aciertos / n:.1%}")
```

Resultados desde todos los estados iniciales (0 a 8)

Estado inicial	Trayectoria	Estado final	Valor final
0	0 → 1 → 2	2	7
1	1 → 2	2	7
2	2	2	7
3	3 → 2	2	7
4	4 → 5 → 6	6	9
5	5 → 6	6	9
6	6	6	9
7	7 → 6	6	9
8	8 → 7 → 6	6	9

Ejecución



Punto 3

A. Componente matemático y conceptual

Fórmulas: $\Delta E = E(\text{nuevo}) - E(\text{actual})$, con $E = x^2$. $P = 1$ si $\Delta E \leq 0$; $P = e^{(-\Delta E/T)}$ si $\Delta E > 0$. Se acepta si $u < P$.

1. Casos

Caso A ($3 \rightarrow 2$, T=5, u=0,70)

$\Delta E = 2^2 - 3^2 = 4 - 9 = -5 \leq 0 \rightarrow P = 1$.

Respuesta: acepta.

Caso B ($2 \rightarrow 3$, T=5, u=0,30)

$\Delta E = 9 - 4 = +5$.

$P = e^{(-5/5)} = e^{-1} = 0{,}3679$; $u = 0{,}30 < 0{,}3679$.

Respuesta: acepta.

Caso C ($2 \rightarrow 3$, T=1, u=0,30)

$\Delta E = +5$.

$P = e^{(-5/1)} = e^{-5} = 0{,}0067$; $u = 0{,}30 > 0{,}0067$.

Respuesta: rechaza.

2. Temperatura

T alta $\rightarrow$ P≈1 (exploración). T baja $\rightarrow$ P≈0 (solo mejoras).

3. Enfriamiento rápido

T $\rightarrow$ 0 antes de explorar: deja de aceptar peores y queda en un óptimo local.

4. Comparación con ascenso de colinas

Ascenso de colinas: solo acepta mejoras. Temple simulado: acepta peores con P = e^(−ΔE/T), por eso escapa de óptimos locales.

B. Implementación en Python

Parámetros: $x_0 = 5$, $T_0 = 10$, enfriamiento geométrico T(k+1) = 0,90·T(k), candidato aleatorio entre x−1 y x+1 válidos, 100 iteraciones y semilla 42.

```
import math
import random
import matplotlib.pyplot as plt


def energia(x):
    return x ** 2


def casos_manuales():
    print("Casos A, B, C")
    casos = [("A", 3, 2, 5, 0.70), ("B", 2, 3, 5, 0.30), ("C", 2, 3, 1, 0.30)]
    for nombre, act, cand, T, u in casos:
        dE = energia(cand) - energia(act)
        p = 1.0 if dE <= 0 else math.exp(-dE / T)
        decision = "ACEPTA" if (dE <= 0 or u < p) else "RECHAZA"
        print(f"  {nombre}: dE={dE:+d}  P={p:.4f}  u={u:.2f}  -> {decision}")
    print()


def temple_simulado(x0=5, T0=10.0, alfa=0.90, iteraciones=100, semilla=42):
    random.seed(semilla)
    x, T = x0, T0
    mejor = x
    hist_T, hist_E = [], []
    for _ in range(iteraciones):
        vecinos = [v for v in (x - 1, x + 1) if -5 <= v <= 5]
        cand = random.choice(vecinos)
        dE = energia(cand) - energia(x)
        if dE <= 0 or random.random() < math.exp(-dE / T):
            x = cand
        if energia(x) < energia(mejor):
            mejor = x
        hist_T.append(T)
        hist_E.append(energia(x))
        T *= alfa
    return mejor, hist_T, hist_E


if __name__ == "__main__":
    casos_manuales()
    mejor, hist_T, hist_E = temple_simulado()

    print(f"Mejor solución: x = {mejor}")
    print(f"Energía mínima: E = {energia(mejor)}")
    print("Historial de temperatura:", [round(t, 4) for t in hist_T])
    print("Historial de energía:", hist_E)

    # --- Generación y guardado de la gráfica ---
    plt.figure(figsize=(10, 5))

    # Subgráfica 1: Evolución de la Energía
    plt.subplot(1, 2, 1)
    plt.plot(hist_E, color='purple', drawstyle='steps-post', linewidth=1.5)
    plt.title('Evolución de la Energía E(x)')
    plt.xlabel('Iteración')
    plt.ylabel('Energía E(x)')
    plt.grid(True, linestyle='--', alpha=0.6)

    # Subgráfica 2: Evolución de la Temperatura
    plt.subplot(1, 2, 2)
    plt.plot(hist_T, color='darkorange', linewidth=1.5)
    plt.title('Enfriamiento Geométrico (T)')
    plt.xlabel('Iteración')
```

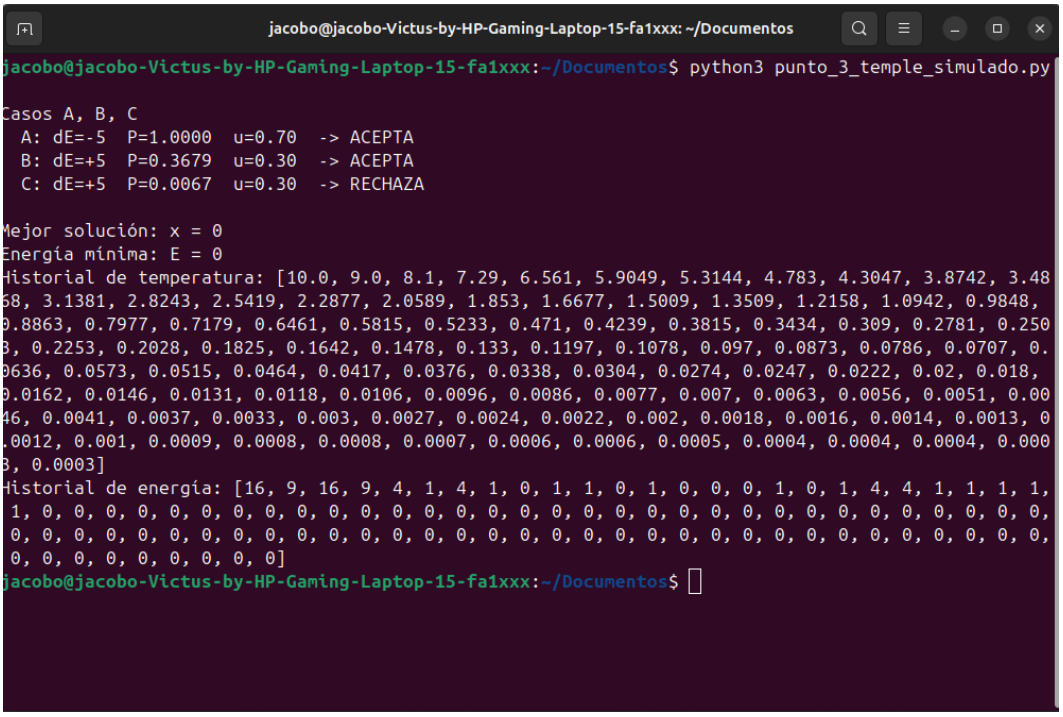
```
    plt.ylabel('Temperatura T')
    plt.grid(True, linestyle='--', alpha=0.6)

    plt.tight_layout()
    plt.savefig('temple_simulado_grafica.png', dpi=300)
    print("\nGráfica guardada exitosamente como 'temple_simulado_grafica.png'.")
    plt.show()
```

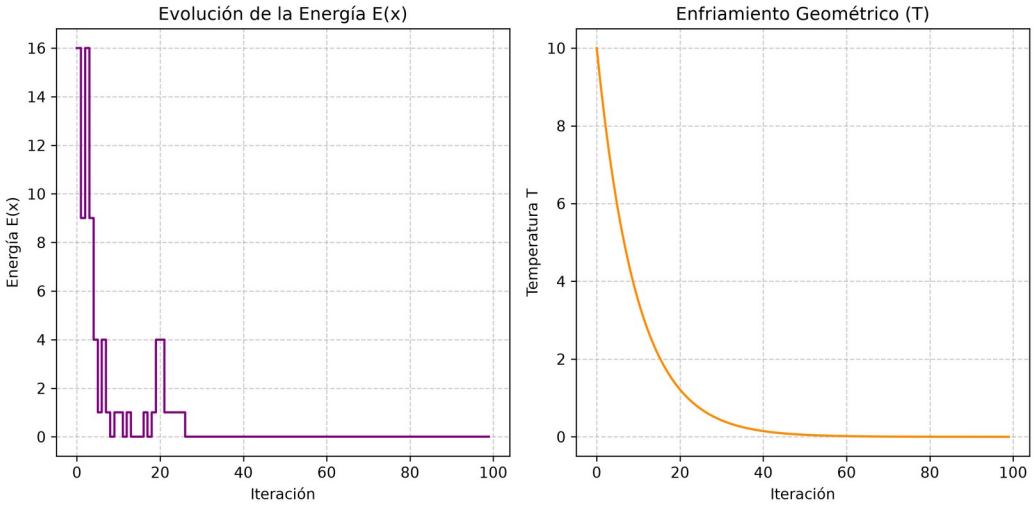
Resultados (semilla 42)

Mejor solución: x = 0. Energía mínima: E = 0. Temperatura: T(k) = 10·0,9^(k−1), con T(100) ≈ 0,0003. La energía del estado actual llega a 0 en la iteración 9 y permanece en 0 desde la iteración 27. El historial completo de temperatura y energía se muestra en la ejecución y en la gráfica.

Ejecución



Gráfica de energía y temperatura



Punto 4

A. Componente matemático y conceptual

Fórmulas: η_Aj = 1/d_Aj. p_Aj = (τ_Aj^α · η_Aj^β) / Σ_k (τ_Ak^α · η_Ak^β), con α = 1 y β = 2. τ_Aj(nueva) = (1 − ρ) · τ_Aj + Δτ_Aj, con ρ = 0,2; Δτ_AB = Q/L con Q = 10 y L = 10.

1. Peso y probabilidad de cada alternativa

Información heurística (η = 1/d)

η_AB = 1/2 = 0,5; η_AC = 1/4 = 0,25; η_AD = 1/5 = 0,2.

Peso $w_{Aj} = \tau_{Aj}^{1} \cdot \eta_{Aj}^{2}$

w_AB = 1 · (0,5)² = 1 · 0,25 = 0,25

w_AC = 2 · (0,25)² = 2 · 0,0625 = 0,125

w_AD = 1 · (0,2)² = 1 · 0,04 = 0,04

Suma de pesos: 0,25 + 0,125 + 0,04 = 0,415

Probabilidades normalizadas

p_AB = 0,25 / 0,415 ≈ 0,6024 (60,24 %)

p_AC = 0,125 / 0,415 ≈ 0,3012 (30,12 %)

p_AD = 0,04 / 0,415 ≈ 0,0964 (9,64 %)

Comprobación: 0,6024 + 0,3012 + 0,0964 = 1,0000.

2. Alternativa más probable

Respuesta: A → B, con p ≈ 0,602. Su elección no está garantizada: la regla de ACO es probabilística (sorteo ponderado), por lo que existe una probabilidad de 1 − 0,6024 ≈ 0,398 (casi 40 %) de que la hormiga escoja C o D. Esa aleatoriedad permite explorar rutas alternativas.

3. Actualización de las feromonas

Depósito

$\Delta\tau_{AB} = Q / L = 10 / 10 = 1$; $\Delta\tau_{AC} = 0$; $\Delta\tau_{AD} = 0$.

Nuevas feromonas (factor de persistencia $1 - \rho = 0{,}8$)

$\tau_{AB}(\text{nueva}) = 0{,}8 \cdot 1 + 1 = 1{,}8$

$\tau_{AC}(\text{nueva}) = 0{,}8 \cdot 2 + 0 = 1{,}6$

$\tau_{AD}(\text{nueva}) = 0{,}8 \cdot 1 + 0 = 0{,}8$

Arista	τ anterior	Evaporación (0,8·τ)	Δτ	τ nueva
A → B	1	0,8	1	**1,8**
A → C	2	1,6	0	**1,6**
A → D	1	0,8	0	**0,8**

Respuesta: $\tau_{AB} = 1{,}8$; $\tau_{AC} = 1{,}6$; $\tau_{AD} = 0{,}8$. Aunque C tenía más feromona inicial, B pasa a tener la mayor porque recibió refuerzo mientras las otras aristas solo se evaporaron.

4. Evaporación y dominancia de la feromona

Función de la evaporación: reduce gradualmente la feromona de todas las aristas, de modo que el sistema "olvida" rutas antiguas o poco útiles. Evita que la feromona se acumule sin límite y que el algoritmo quede atado a decisiones tempranas, manteniendo la exploración y la corrección de elecciones subóptimas.

Si la feromona dominara excesivamente: el término τ^{α} pesaría mucho más que el heurístico η^{β} y las probabilidades se concentrarían en una sola arista. Se genera un círculo vicioso (más hormigas la eligen → más feromona → más hormigas la eligen) que provoca convergencia prematura o estancamiento: todas las hormigas siguen la misma ruta, se pierde la exploración y el algoritmo puede quedar atrapado en un óptimo local.

B. Implementación en Python

Parámetros: 10 hormigas por iteración, 20 iteraciones, $\alpha = 1$, $\beta = 2$, $\rho = 0{,}2$, $Q = 10$, τ inicial = 1 en cada ruta y semilla 42. Rutas: A-B-Z (L = 6), A-C-Z (L = 8) y A-D-Z (L = 10).

```
import random

import matplotlib
matplotlib.use("Agg")  # backend sin ventana: guarda la figura directamente
import matplotlib.pyplot as plt

# Parámetros
RUTAS = {"A-B-Z": 6, "A-C-Z": 8, "A-D-Z": 10}  # nombre -> longitud total L
N_HORMIGAS = 10
N_ITERACIONES = 20
ALPHA = 1
BETA = 2
RHO = 0.2
Q = 10
TAU_INICIAL = 1.0
SEMILLA = 42


def calcular_probabilidades(tau, rutas, alpha, beta):
```

```
    """Devuelve {ruta: probabilidad} normalizando tau^alpha * (1/L)^beta."""
    pesos = {r: (tau[r] ** alpha) * ((1.0 / L) ** beta) for r, L in rutas.items()}
    total = sum(pesos.values())
    return {r: w / total for r, w in pesos.items()}


def elegir_rutas(probs, n_hormigas):
    """Cada hormiga elige una ruta con las probabilidades dadas."""
    nombres = list(probs.keys())
    pesos = [probs[r] for r in nombres]
    return random.choices(nombres, weights=pesos, k=n_hormigas)


def actualizar_feromonas(tau, elecciones, rutas, rho, q):
    """Evapora todas las rutas y luego suma el depósito Q/L de cada hormiga."""
    for r in tau:                       # 3. evaporación
        tau[r] *= (1.0 - rho)
    for r in elecciones:                # 4. depósito por hormiga
        tau[r] += q / rutas[r]


def ejecutar_aco():
    """Corre el algoritmo completo y devuelve frecuencias, tau final e historial."""
    random.seed(SEMILLA)
    tau = {r: TAU_INICIAL for r in RUTAS}
    frecuencias = {r: 0 for r in RUTAS}
    historial = {r: [tau[r]] for r in RUTAS}  # incluye iteración 0 (estado inicial)

    for it in range(1, N_ITERACIONES + 1):
        probs = calcular_probabilidades(tau, RUTAS, ALPHA, BETA)   # 1.
        elecciones = elegir_rutas(probs, N_HORMIGAS)               # 2.
        for r in elecciones:
            frecuencias[r] += 1
        actualizar_feromonas(tau, elecciones, RUTAS, RHO, Q)       # 3. y 4.
        for r in RUTAS:
            historial[r].append(tau[r])

    return frecuencias, tau, historial


def graficar(historial, archivo="aco_feromonas.png"):
    """Grafica la evolución de tau de cada ruta y guarda la imagen."""
    plt.figure(figsize=(8, 5))
    for r, valores in historial.items():
        plt.plot(range(len(valores)), valores, marker="o", markersize=4,
                 label=f"{r} (L={RUTAS[r]})")
    plt.xlabel("Iteración")
    plt.ylabel("Feromona τ")
    plt.title("Evolución de la feromona por ruta (ACO)")
    plt.xticks(range(0, N_ITERACIONES + 1, 2))
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.savefig(archivo, dpi=150)
    plt.close()


if __name__ == "__main__":
    frecuencias, tau_final, historial = ejecutar_aco()
    total = N_HORMIGAS * N_ITERACIONES

    print("Frecuencias de elección (total = %d elecciones):" % total)
    for r, f in frecuencias.items():
        print(f"  {r}: {f:>3}  ({f / total:.1%})")

    print("\nFeromonas finales:")
    for r, t in tau_final.items():
        print(f"  {r}: {t:.4f}")

    mejor_observada = max(frecuencias, key=frecuencias.get)
    print(f"\nMejor ruta observada (más elegida): {mejor_observada} "
          f"(L = {RUTAS[mejor_observada]})")
    print(f"Ruta más corta del problema: {min(RUTAS, key=RUTAS.get)} "
          f"(L = {min(RUTAS.values())})")

    graficar(historial)
```

```
    print("\nGráfica guardada como 'aco_feromonas.png'")
```

Resultados (semilla 42)

En las 200 elecciones (10 hormigas × 20 iteraciones), la ruta A-B-Z fue elegida 187 veces (93,5 %), A-C-Z 12 veces (6,0 %) y A-D-Z una sola vez (0,5 %). La mejor ruta observada es A-B-Z (L = 6), que coincide con la ruta más corta del problema.

Ruta	L	Frecuencia	%	τ final
A-B-Z	6	187	93,5	79,2315
A-C-Z	8	12	6,0	2,3580
A-D-Z	10	1	0,5	0,0259

La feromona de A-B-Z crece porque es la ruta con mayor probabilidad inicial y además recibe el mayor depósito por hormiga (Q/L = 10/6 ≈ 1,67). Esa ventaja se refuerza en cada iteración, mientras que A-C-Z y A-D-Z reciben poco depósito y la evaporación reduce su feromona.

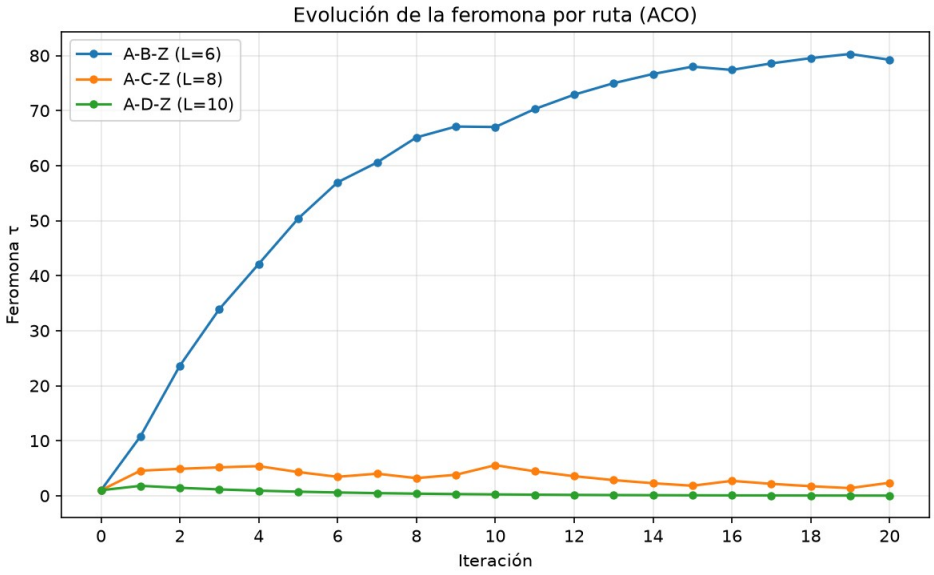
Ejecución

```
(venv) jacobo@jacobo-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~/Documentos/parcialIA
2$ python3 punto_4_aco.py
Frecuencias de elección (total = 200 elecciones):
  A-B-Z: 187  (93.5%)
  A-C-Z:  12  (6.0%)
  A-D-Z:   1  (0.5%)

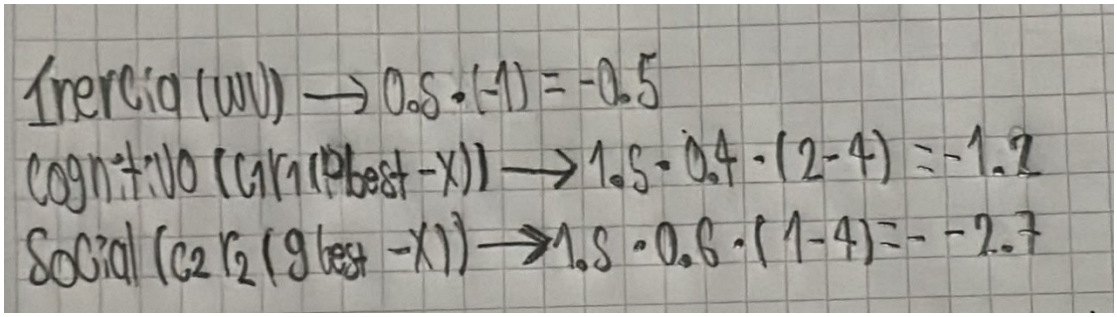
Feromonas finales:
  A-B-Z: 79.2315
  A-C-Z: 2.3580
  A-D-Z: 0.0259

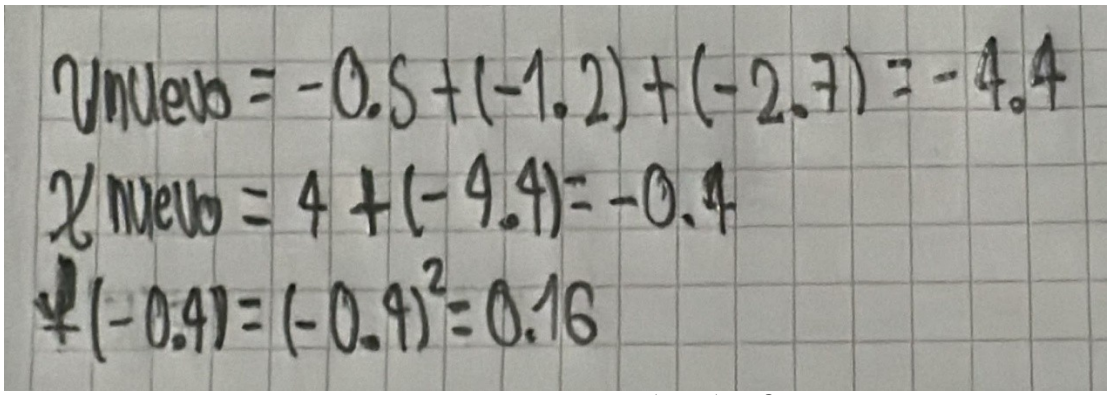
Mejor ruta observada (más elegida): A-B-Z (L = 6)
Ruta más corta del problema: A-B-Z (L = 6)
```

Gráfica de evolución de las feromonas



Punto 5

1. 

2. 

3. **Para p_{best}:** La aptitud anterior de la partícula era $f(p_{best}) = 2^2 = 4$, la nueva aptitud es 0.16 y como $0.16 < 4$ y estamos minimizando si se debe actualizar p_{best} a -0.4

Para g_{best}: La mejor posición global actual tenía una aptitud de $f(g_{best}) = 1^2 = 1$, como la nueva aptitud de la partícula es 0.16, y $0.16 < 1$, sí debe actualizarse g_{best} al nuevo valor -0.4

4. **Mejor posición personal:** Es la mejor posición individual que ha encontrado una partícula específica a lo largo de su propia historia de exploración
   Mejor posición global: Es la mejor posición encontrada por cualquier partícula de todo el enjambre hasta el momento actual
5. En PSO, la información se comparte de forma numérica y centralizada mediante la guía de las mejores posiciones (p_{best} y g_{best}), atrayendo directamente a las partículas hacia coordenadas espaciales prometedoras

   En ACO, el intercambio de información es indirecto a través del depósito físico de feromonas en las rutas, atrayendo probabilísticamente a las hormigas hacia los caminos que colectivamente han demostrado ser más eficientes

Ejecución

